



5.2 Local Storage

Ionic tiene un módulo para poder utilizar nuestro dispositivo para almacenar localmente información: Ionic Storage.

Almacenar datos físicamente en mi dispositivo, aunque es factible no es la mejor opción porque si se da el caso de que android necesitara espacio, el local storage es lo primero que borra.

Para utilizarlo instalamos `npm install @ionic/storage`

Al usar angular debemos instalar el módulo `storage-angular`

```
npm install @ionic/storage-angular
```

Una vez instalado, añadimos el correspondiente `import {IonicStorageModule}` en el `app.module` y en los imports añadimos `IonicStorageModule.forRoot()`. También debemos añadir en providers la clase `Store`

Para utilizar el storage, inyectamos la clase `Storage` dónde lo necesitamos.

Podemos optar por generar una base de datos local cuándo grabemos en el storage. Esta bbdd está implementada en SQLite y necesitamos instalar un plugin de cordova. Si no usamos capacitor, hay que instalar el cli de cordova

```
sudo npm i -g cordova
```

Una vez instalado

Installation

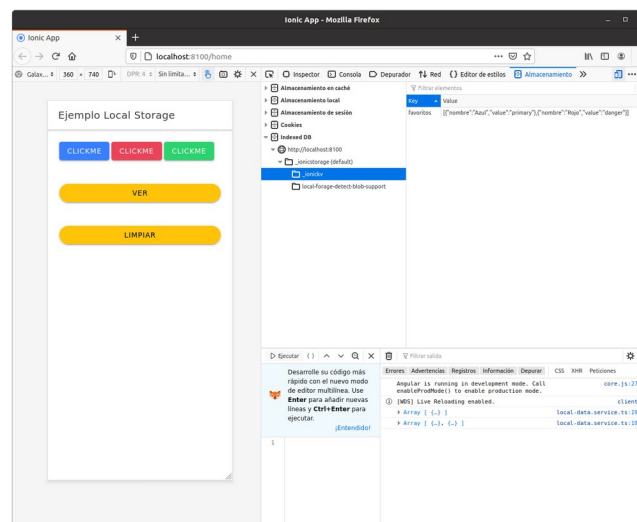
```
# If using Cordova, install the plugin using
ionic cordova plugin add cordova-sqlite-storage
# If using Capacitor, install the plugin using
npm install cordova-sqlite-storage

# Then, install the npm library
npm install localforage-cordova-sqlitedriver
```

Una vez hecha la instalación:

- Para crear el almacenamiento `storage.create()` antes de empezar a almacenar
- Para guardar valores en el `storage.set(nombre, valor)` donde nombre será la clave del elemento que almacenamos y valor puede ser de cualquier tipo

Una vez que añadimos un valor podremos ver en el explorador la base de datos que ha creado



- Para recuperar un elemento del “almacenamiento” `storage.get(nombre)`,

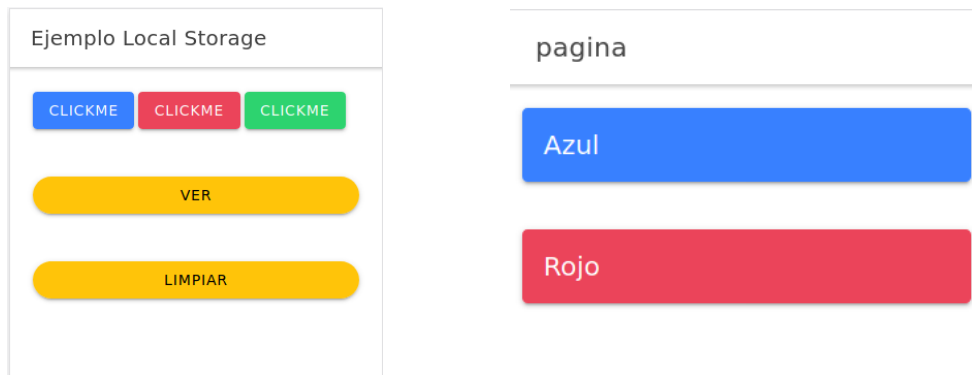
Utilizaremos las funciones `async / await`. Nos permiten escribir código asíncrono que se parece un poco más al síncrono y por tanto nos resulta más fácil de implementar. Están basadas en las llamadas “promesas” (Promise), esto nos proporciona una manera fácil de tratar secuencialmente la asincronía en nuestro código

Si definimos una función asíncrona, usaremos `await` para indicar que cuándo el resultado esté listo será retornado.

```
public async getColores(){
  this.colores= await this.storage.get('favoritos');
  if (!this.colores)
    this.colores=[];
  return this.colores;
}
```

Ese código nos retorna un `Promise<IColor[]>`

Ejemplo: Aplicación que muestra tres botones de color azul, rojo y verde. Guardamos las sucesivas pulsaciones que vamos haciendo en el storage de manera que al pulsar un cuarto botón cargamos otra página en la que mostramos la secuencia de colores que hemos pulsado



- El botón limpiar lo usaremos para reiniciar el storage : `this.storage.clear()`
- Para borrar un elemento de la bbdd local `this.storage.remove(nombre)` dónde nombre será el del elemento que queremos borrar. En nuestro ejemplo, al ser un array si pasamos el nombre del array lo borraríamos entero, por lo que tendremos que implementar el código para que se borre el que queramos (utilizaremos javascript)